MediBook — Integrated System Milestone

IT342 Capstone Project — Integrated System Milestone

Student: Amihan Gyle Amihan
Section: IT342
Date: July 15, 2026

GitHub Repository: https://github.com/amihangyle/MediBook

---

1. Development Progress Summary

MediBook is a three-tier medical appointment booking system with a Spring Boot backend, React web dashboard, and Android Kotlin mobile app, backed by Supabase PostgreSQL.

This milestone phase focused on completing the remaining supporting features (patient profile management, appointment cancellation, schedule management), integrating them across all system components (backend API, web dashboard, mobile app), implementing comprehensive testing (37 unit tests + 4 E2E integration tests), and resolving security configuration issues.

Key Achievements

- **41 tests passing** (37 unit + 4 E2E integration)
- **9 separate Git commits** with clear, meaningful messages
- **3 complete end-to-end workflow tests** including invalid action and validation scenarios
- **3 supporting features** fully implemented across backend, web, and mobile
- **Security bug fixed** that was blocking patient self-service access

---

2. Newly Implemented Features

Feature 1: Patient Self-Service Profile Management

Functional Requirement: FR-001 (Patient Registration) extended to include profile management.

Backend (Spring Boot): - `GET /api/patients/me` — Returns the authenticated patient's profile (name, email, contact, address) - `PUT /api/patients/me` — Updates patient's own profile with `@NotBlank` validation on fullName - `PatientRequest` DTO with Bean Validation annotations - `PatientService.getPatientByUserId()` with ownership validation - `PatientController` using `@AuthenticationPrincipal` for secure identity resolution

Mobile (Kotlin/Android): - `PatientApiService` interface with Retrofit for GET/PUT `/api/patients/me` - `PatientResponse` and `PatientRequest` data models with Gson serialization - `ProfileEditActivity` with form fields: Full Name (editable), Contact Number (editable), Address (editable), Email (read-only) - "My Profile" card added to dashboard with navigation - `RetrofitClient` updated with `patientApi` instance - `AndroidManifest.xml` updated with `ProfileEditActivity`

Integration: The mobile app communicates with the backend through the `PatientApiService`. When the patient saves changes, the request is sent via `PUT /api/patients/me` with a Bearer token. The backend validates the token via `JwtAuthFilter`, resolves the `User` entity via `@AuthenticationPrincipal`, and updates the patient record in the database.

Feature 2: Patient Appointment Cancellation

Functional Requirement: FR-004 (Appointment Confirmation) extended with patient-initiated cancellation.

Backend (Spring Boot): - `PATCH /api/appointments/{id}/cancel` — Patients can cancel their own PENDING or CONFIRMED appointments - Ownership validation: checks `appointment.getPatient().getPatientId()` matches the authenticated patient - Reopens the `DoctorSchedule` slot (`setIsAvailable(true)`) when cancelled - Sends email notification on cancellation via `EmailService` - State machine enforced: COMPLETED appointments cannot be cancelled (409 Conflict)

Mobile (Kotlin/Android): - `cancelAppointment()` added to `AppointmentApiService` (`PATCH /api/appointments/{id}/cancel`) - Red "Cancel Appointment" button in the appointment detail dialog - Button only visible for PENDING or CONFIRMED appointments - Confirmation dialog ("Are you sure?") before cancellation to prevent accidental actions - Auto-refreshes appointment list after successful cancellation - Handles 401 unauthorized with redirect to LoginActivity

Integration: The mobile app's cancel button triggers a PATCH request to the backend. The backend's `JwtAuthFilter` validates the JWT token, the `SecurityConfig` enforces PATIENT role for this endpoint, and `AppointmentService.cancelMyAppointment()` validates ownership before transitioning the appointment to CANCELLED status.

Feature 3: Schedule Management (Web Dashboard)

Functional Requirement: FR-005 (Schedule Management) — previously only had "create" functionality.

Backend (Spring Boot): - `PUT /api/schedules/{scheduleId}` already existed but was unused by the frontend - `DoctorScheduleService.updateSchedule()` with overlap prevention (BR-003)

Web Dashboard (React): - `ManageSchedulesPage.jsx` — New page with table listing all schedule slots - Columns: Doctor Name, Start Time, End Time, Status (Available/Booked), Action (Edit) - Inline edit form for updating start/end time of existing slots - `updateSchedule()` API function added to `scheduleService.js` - Route `/schedules` added to `AppRoutes.jsx`, protected for ADMIN and STAFF roles - `btn-outline` CSS utility class added for secondary actions

Integration: The web dashboard fetches all schedules via `GET /api/schedules` and displays them in a table. When staff clicks "Edit", an inline form appears with the current values. On save, the frontend sends `PUT /api/schedules/{id}` to the backend, which validates the time range and checks for overlapping slots before updating.

---

3. System Component Integration

Backend ↔ Mobile Integration

Endpoint	Mobile Component	Backend Component	Data Exchan
`GET /api/patients/me`	`PatientApiService.getMyProfile()`	`PatientController.getMyProfile()`	JSON: `PatientResp`
`PUT /api/patients/me`	`PatientApiService.updateMyProfile()`	`PatientController.updateMyProfile()`	JSON: `PatientRequ` `PatientResp`
`PATCH /api/appointments/{id}/cancel`	`AppointmentApiService.cancelAppointment()`	`AppointmentController.cancelMyAppointment()`	Path param + `Appointment`

Backend ↔ Web Integration

Endpoint	Web Component	Backend Component	Data Exchange
`GET /api/schedules`	`ManageSchedulesPage` via `getSchedules()`	`DoctorScheduleController.getAvailableSchedules()`	JSON: `List<DoctorScheduleResponse>`
`PUT /api/schedules/{id}`	`ManageSchedulesPage` via `updateSchedule()`	`DoctorScheduleController.updateSchedule()`	JSON: `DoctorScheduleRequest` → `DoctorScheduleResponse`

Authentication Flow

All three components share the same JWT-based authentication: 1. User authenticates via `POST /api/auth/login` 2. Backend returns access + refresh tokens 3. Mobile stores tokens via `TokenManager`; Web stores via `AuthContext` 4. Subsequent requests include `Authorization: Bearer {token}` header 5. Backend's `JwtAuthFilter` validates token, loads `UserDetails`, sets `SecurityContext` 6. `@AuthenticationPrincipal` resolves the `User` entity in controllers

Database Consistency

All features use the same PostgreSQL database through JPA/Hibernate: - Patient profile updates go through `PatientRepository.save()` within `@Transactional` boundaries - Appointment cancellation atomically updates both the `Appointment` status and `DoctorSchedule.isAvailable` - Schedule overlap prevention uses database-level `@Lock(PESSIMISTIC_WRITE)` to prevent race conditions

---

4. Screenshots

> ***Note:*** *Add your screenshots to this section before submitting the PDF.*

Screenshot 1: Web Dashboard — Schedule Management Page

[Insert screenshot of ManageSchedulesPage showing schedule table with Edit buttons]

Screenshot 2: Web Dashboard — Schedule Edit Form

[Insert screenshot of inline edit form with datetime pickers]

Screenshot 3: Mobile — Appointment Detail with Cancel Button

[Insert screenshot of appointment detail dialog showing red Cancel button for PENDING appointment]

Screenshot 4: Mobile — Cancel Confirmation Dialog

[Insert screenshot of "Are you sure?" confirmation dialog]

Screenshot 5: Mobile — Patient Profile Edit Screen

[Insert screenshot of ProfileEditActivity with form fields]

Screenshot 6: Mobile — Dashboard with My Profile Card

[Insert screenshot of dashboard showing Browse Doctors, My Appointments, and My Profile cards]

---

5. End-to-End Workflow Test Results

Test Suite Summary

```
Tests run: 41, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Test Class	Tests	Status
AppointmentServiceTest (State Machine)	13	All Pass
HealthRecordServiceTest (IDOR & Business Logic)	10	All Pass
DoctorScheduleServiceTest (Overlap Prevention)	8	All Pass
RateLimitServiceTest (Brute Force Protection)	6	All Pass
E2EWorkflowTest (Integration)	4	All Pass

E2E Workflow 1: Patient Books → Staff Confirms → Patient Views (Happy Path)

Description: Tests the complete appointment lifecycle from booking through confirmation to viewing.

Test method: `fullAppointmentLifecycle()`

Steps: 1. Patient books an appointment against schedule slot 1 → **201 CREATED** - Response: `{"appointmentId":1, "status":"PENDING", "patientName":"Juan Dela Cruz"}` 2. Staff confirms the appointment → **200 OK** - Response: `{"status":"CONFIRMED"}` 3. Patient views their appointments → **200 OK** - Response: `[{"appointmentId":1, "status":"CONFIRMED"}]`

Result: PASS

E2E Workflow 2: Invalid State Transition (Validation Scenario)

Description: Tests that the state machine rejects invalid transitions — specifically, trying to confirm an already COMPLETED appointment.

Test method: `invalidStateTransitionRejected()`

Steps: 1. Patient books an appointment → **201 CREATED** 2. Staff confirms → **200 OK** (PENDING → CONFIRMED) 3. Staff completes → **200 OK** (CONFIRMED → COMPLETED) 4. Staff tries to confirm again → **409 CONFLICT** - Error: `"Cannot transition appointment from COMPLETED to CONFIRMED."`

Result: PASS — Demonstrates invalid action validation as required by the rubric.

E2E Workflow 3: Patient Profile Validation

Description: Tests both successful profile update and validation rejection for blank required fields.

Test method: `patientUpdatesProfile()` and `blankNameRejected()`

Steps (Validation Success): 1. Patient updates profile with valid data → **200 OK** - Response: `{"fullName":"Juan Dela Cruz Updated"}`

Steps (Validation Failure): 2. Patient tries to update with blank fullName → **400 BAD_REQUEST** - Error: `"Full name is required"`

Result: PASS — Demonstrates input validation and proper error responses.

Unit Test Coverage

Test Category	What is Tested
State Machine (13 tests)	Book success, patient not found, slot unavailable, all valid transitions (PENDING→CONFIRMED, PENDING→CANCELLED, CONFIRMED→COMPLETED, CONFIRMED→CANCELLED), all invalid transitions, delete with slot reopen, delete blocked by health record

Test Category	What is Tested
IDOR Protection (10 tests)	Patient A cannot access Patient B's records, Doctor A cannot access Doctor B's records, non-CONFIRMED appointment rejection, duplicate record prevention, update ownership checks
Overlap Prevention (8 tests)	Create with overlap rejected, create success, doctor not found, invalid times, equal times, update overlap, update success, update not found
Rate Limiting (6 tests)	New key creation, blocks after 5 attempts, below threshold allowed, reset clears count, independent keys, thread-safe concurrent access

---

6. Integration Issues Encountered and Solutions Applied

Issue 1: Spring Boot 4.x Package Renames

Problem: Spring Boot 4.1.0 moved `@WebMvcTest` and `@AutoConfigureMockMvc` to new packages under `org.springframework.boot.webmvc.test.autoconfigure`. **Solution:** Updated all test imports to use the new package paths.

Issue 2: `@AuthenticationPrincipal User` Causes NPE in Tests

Problem: The `User` entity implements `UserDetails`. `SecurityMockMvcRequestPostProcessors.user("username")` creates a plain `UserDetails` string principal, not the `User` entity, causing `NullPointerException` in controllers. **Solution:** Load actual `User` entities from the database via `UserRepository` and pass them: `SecurityMockMvcRequestPostProcessors.user(patientUser)`.

Issue 3: Standalone MockMvc Cannot Resolve `@AuthenticationPrincipal`

Problem: Standalone MockMvc (without `@SpringBootTest`) doesn't register the `AuthenticationPrincipalArgumentResolver`. **Solution:** Switched to `@SpringBootTest` + `@AutoConfigureMockMvc` with H2 in-memory database for full application context.

Issue 4: Spring Boot 4.x Uses Jackson 3, Not Jackson 2

Problem: Spring Boot 4.x auto-configures `tools.jackson.databind.json.JsonMapper`, but the test tried to autowire `com.fasterxml.jackson.databind.ObjectMapper`. **Solution:** Created the `ObjectMapper` manually in the test instead of autowiring it.

Issue 5: EmailConfig Bean Fails Without SMTP Server

Problem: `@SpringBootTest` tries to create `JavaMailSender` bean, which requires a running SMTP server. **Solution:** Removed `spring.mail.*` properties from test `application.properties` so `EmailConfig` (annotated with `@ConditionalOnProperty`) doesn't activate. `EmailService` gracefully handles the missing mail sender with `@Autowired(required = false)`.

Issue 6: `JwtUtil` Requires `jwt.access-token-ms` Property

Problem: The test `application.properties` initially only had `jwt.secret` and `jwt.expiration`, but `JwtUtil` uses `jwt.access-token-ms` and `jwt.refresh-token-ms`. **Solution:** Added all required JWT properties to the test configuration.

Issue 7: SecurityConfig Rule Ordering Blocks Patient Access

Problem: The `/api/patients/**` matcher was listed before `PUT /api/patients/me`. In Spring Security 6+, the first matching rule wins, so patients were getting 403 on their own profile endpoint. **Solution:** Moved the specific `GET/PUT /api/patients/me` rules before the general `/api/patients/**` rule.

Issue 8: No Schedule Slots in Test Database

Problem: E2E tests needed to book appointments, but the `DataSeeder` only created users, not schedule slots. **Solution:** Updated `DataSeeder` to create 5 available doctor schedule slots and a patient user.

Issue 9: `@Transactional` Test Rollback Causes Cross-Test Isolation Issues

Problem: With `@Transactional` on the test class, each test's data is rolled back. But without it, Workflow 1's booked slot is still consumed when Workflow 2 runs. **Solution:** Removed `@Transactional`, used separate schedule slot IDs per workflow, and used dynamic appointment IDs extracted from POST responses.

---

7. Commit History Table

#	Commit Message	Files Changed	GitHub Link
1	test(backend): add 37 unit tests for appointment, health record, schedule, and rate limiting	5	6ac222c

#	Commit Message	Files Changed	GitHub Link
2	test(backend): add E2E integration tests with H2 in-memory database	4	a1f188f
3	feat(backend): add patient self-service profile endpoints (GET/PUT /api/patients/me)	3	7e523b6
4	feat(backend): add patient cancel appointment endpoint (PATCH /api/appointments/{id}/cancel)	2	9fbe70c
5	fix(security): reorder SecurityConfig matchers and add role-based access rules	1	9141a5f
6	refactor(web): extract DashboardHome, shared form utilities, and fix imports	12	70bddca
7	feat(web): add schedule management page with list view and inline editing	4	c21db2c
8	feat(mobile): add appointment cancellation feature for patients	3	19c87a4
9	feat(mobile): add patient profile view and edit screen	9	024807c

Total commits this milestone: 9
Total files changed: 43
Total lines added: ~1,900+

---

8. Individual Contribution Statement

Amihan Gyle Amihan — Sole developer and contributor.

All source code, tests, documentation, and commits in this milestone were individually authored, tested, and committed by Amihan Gyle Amihan. AI assistance (Claude) was used as a development tool for code generation, debugging, and documentation, under the direct supervision and review of the student. The student understands and can explain all submitted source code.

Work Breakdown

- **Backend API development:** Patient profile endpoints, patient cancel endpoint, security config fixes, DataSeeder updates
- **Backend testing:** 37 unit tests (AppointmentService, HealthRecordService, DoctorScheduleService, RateLimitService), 4 E2E integration tests with full security filter chain
- **Web frontend:** Schedule management page, DashboardHome extraction, shared form utilities, route configuration
- **Mobile development:** Cancel appointment feature, patient profile edit screen, API service interfaces, dashboard integration
- **Infrastructure:** H2 test database configuration, Spring Boot 4.x compatibility fixes, JWT test properties
- **Documentation:** This milestone report